Ho Chi Minh City University of Technology
Faculty of Electrical – Electronics Engineering
Integrated Circuit Design Major



EE3213 - DIGITAL SYSTEM DESIGN AND VERIFICATION

Milestone 1
AES-128 Encryption Accelerator

Instructor: Hieu Nguyen

Email: hieunt@hcmut.edu.vn

October 26, 2026

Contents

1 Introduction **2**

1.1 AES-128 Overview . . . 2

1.2 AES State Representation . . . 2

1.3 AES-128 Encryption Flow . . . 3

1.4 SubBytes . . . 3

1.5 ShiftRows . . . 5

1.6 MixColumns and Finite-Field Arithmetic . . . 5

1.7 Round-Key XOR (AddRoundKey) . . . 6

1.8 AES-128 Key Expansion . . . 7

2 Requirements and Implementation Procedure **9**

2.1 Requirements . . . 9

2.2 Top-Level Interface . . . 10

2.3 AES-128 Hardware Architecture Exploration . . . 10

2.3.1 Iterative Architecture . . . 10

2.3.2 Unrolled Architecture . . . 11

2.3.3 Pipelined Architecture . . . 11

2.4 Throughput and Initiation Interval . . . 11

2.5 Implementation Procedure . . . 11

2.5.1 Step 1 – Estimate Candidate Architectures . . . 11

2.5.2 Step 2 – Select the Architecture and Resource Budget . . . 12

2.5.3 Step 3 – Design the Datapath and Control Unit . . . 12

2.5.4 Design the Key Expansion . . . 13

2.5.5 Pipelining and Control . . . 13

2.5.6 Streaming Interface and Backpressure . . . 14

2.5.7 Simulation and Basic Functional Verification . . . 14

2.5.8 Synthesis and Timing Analysis . . . 14

2.5.9 Optional FPGA Demonstration . . . 15

2.5.10 Reports and Design Defense . . . 15

3 Grading Criteria **16**

3.1 Design Learning Outcomes Assessed . . . 16

1 Introduction

1.1 AES-128 Overview

The Advanced Encryption Standard (AES) is a symmetric-key block cipher standardized by NIST. In this project, only AES-128 encryption is considered. AES-128 encrypts a 128-bit plaintext block P using a 128-bit cipher key K, executes 10 encryption rounds, and produces a 128-bit ciphertext block C:

$$\boxed{C = \text{AES128}(P, K)}$$

where P, K, and C are 128-bit vectors.

In the RTL interface, hexadecimal vectors are written from the most-significant byte to the least-significant byte. For example, `128'h00112233445566778899aabbccddeeff` means that the first AES input byte is `8'h00`, followed by `8'h11`, `8'h22`, and so on.

Example 1.1 (Running AES-128 Example) *The following NIST AES-128 known-answer example is used throughout this introduction:*

$$P = \texttt{128'h00112233445566778899aabbccddeeff}$$
$$K = \texttt{128'h000102030405060708090a0b0c0d0e0f}$$
$$C = \texttt{128'h69c4e0d86a7b0430d8cdb78070b4c55a}$$

The value of C is the expected ciphertext after all ten AES-128 rounds. The intermediate values shown in the following subsections correspond to this same plaintext and key. The running example uses the AES-128 example vector from NIST FIPS 197. Additional intermediate example values are available from the NIST AES Example Values page.

The purpose of this project is not only to produce a functionally correct AES implementation. Students must explore the hardware architecture and select an implementation that satisfies the given throughput and clock constraints while minimizing hardware cost.

1.2 AES State Representation

AES interprets the 128-bit data block as a 4×4 array of bytes called the *State*:

$$\mathbf{S} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

AES fills the State column by column. If the input byte sequence is $p_0, p_1, \ldots, p_{15}$, then:

$$\boxed{s_{r,c} = p_{4c+r}}$$

For the running example:

$$P = \texttt{00112233445566778899aabbccddeeff}$$

therefore the plaintext State is:

$$\mathbf{S}_P = \begin{bmatrix} 00 & 44 & 88 & cc \\ 11 & 55 & 99 & dd \\ 22 & 66 & aa & ee \\ 33 & 77 & bb & ff \end{bmatrix}$$

The 128-bit key is mapped using the same byte ordering. For:

$$K = \texttt{000102030405060708090a0b0c0d0e0f}$$

the initial key State is:

$$\mathbf{K}_0 = \begin{bmatrix} 00 & 04 & 08 & 0c \\ 01 & 05 & 09 & 0d \\ 02 & 06 & 0a & 0e \\ 03 & 07 & 0b & 0f \end{bmatrix}$$

For an RTL port declared as `logic [127:0] i_plaintext`, one convenient convention is:

$$\texttt{i_plaintext[127:120]} = p_0, \quad \texttt{i_plaintext[119:112]} = p_1, \quad \ldots \quad , \texttt{i_plaintext[7:0]} = p_{15}$$

Students may use another internal organization, but the byte ordering at the external interface must be documented and used consistently throughout the design and testbench.

1.3 AES-128 Encryption Flow

AES-128 encryption is most clearly described as an initial round-key XOR, a nine-iteration main loop, and one final round. The following pseudocode defines the encryption sequence used in this project:

```
State ← Plaintext
State ← State ⊕K_0
for r = 1 to 9 do
    State ← SubBytes(State)
    State ← ShiftRows(State)
    State ← MixColumns(State)
    State ← State ⊕ K_r
end for
Final round
    State ← SubBytes(State)
    State ← ShiftRows(State)
    State ← State ⊕ K_10
Ciphertext ← State
```

Here, K_0 is the initial round key, K_r is the round key used in main round r, and K_{10} is the final round key. The main loop is executed exactly nine times. The final round omits *MixColumns*.

The following subsections use the same running example to illustrate the transformations appearing in this pseudocode.

1.4 SubBytes

The *SubBytes* transformation applies the AES substitution box (S-box) independently to all 16 bytes of the State. For Round 1 of the running example, the input to SubBytes is the State obtained after the initial round-key XOR:

$$\texttt{00102030405060708090a0b0c0d0e0f0}$$

Using the AES column-major State convention, this 128-bit value is represented as:

$$\mathbf{State}_{in} = \begin{bmatrix} 00 & 40 & 80 & c0 \\ 10 & 50 & 90 & d0 \\ 20 & 60 & a0 & e0 \\ 30 & 70 & b0 & f0 \end{bmatrix}.$$

SubBytes replaces each byte independently:

$$s'_{r,c} = \mathrm{SBox}(s_{r,c}).$$

For any byte written as

$$x = \texttt{0xXY},$$

the upper hexadecimal digit X selects the S-box row and the lower hexadecimal digit Y selects the S-box column:

$$\boxed{\mathrm{SBox}(\texttt{0xXY}) = \mathrm{SBox}[X][Y]}.$$

For example, the first column of $\mathbf{State}_{in}$ is

$$\begin{bmatrix} 00 \\ 10 \\ 20 \\ 30 \end{bmatrix}$$

and the byte substitutions are:

$$\begin{aligned} \mathrm{SBox}(\texttt{00}) &= \texttt{63}, \\ \mathrm{SBox}(\texttt{10}) &= \texttt{ca}, \\ \mathrm{SBox}(\texttt{20}) &= \texttt{b7}, \\ \mathrm{SBox}(\texttt{30}) &= \texttt{04}. \end{aligned}$$

Applying the same rule to all 16 bytes gives:

$$\mathbf{State}_{SB} = \begin{bmatrix} 63 & 09 & cd & ba \\ ca & 53 & 60 & 70 \\ b7 & d0 & e0 & e1 \\ 04 & 51 & e7 & 8c \end{bmatrix}.$$

Serializing the State column by column produces:

$$\boxed{\texttt{63cab7040953d051cd60e0e7ba70e18c}}.$$

The AES S-box is a fixed nonlinear mapping. Algebraically, it is constructed by first computing the multiplicative inverse of the input byte in $GF(2^8)$ using the irreducible polynomial

$$m(x) = x^8 + x^4 + x^3 + x + 1,$$

which corresponds to `0x11B`, followed by an affine transformation. The input `00` is mapped to `00` for the inversion step. If the inverse byte is $b = (b_7, \ldots, b_0)$, the S-box output bits are:

$$y_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i$$

for $i = 0, \ldots, 7$, where $c = \texttt{8'h63}$ and bit index 0 denotes the least-significant bit.

For RTL implementation, students may implement the standard AES S-box as combinational logic, a 256-entry lookup table, or a ROM. Regardless of implementation, the mapping must exactly match the AES S-box defined in NIST FIPS 197.

A complete 128-bit State contains 16 bytes; therefore, one full-state SubBytes operation requires 16 byte substitutions. From a hardware-design perspective, the S-box organization creates an important area-throughput trade-off:

- 16 parallel S-box units can substitute the complete State in one cycle
- fewer S-box units can be shared across multiple cycles to reduce area
- a lookup-table or ROM implementation may have different timing and area from a logic-based implementation

Students must justify the selected S-box implementation and the number of S-box units in terms of area, critical path, latency, and initiation interval.

1.5 ShiftRows

The *ShiftRows* transformation cyclically shifts the rows of the State:

- Row 0: no shift
- Row 1: shift left by one byte
- Row 2: shift left by two bytes
- Row 3: shift left by three bytes

For the running example, the Round-1 SubBytes output

`63cab7040953d051cd60e0e7ba70e18c`

corresponds to the State matrix:

$$\mathbf{State}_{SB} = \begin{bmatrix} 63 & 09 & cd & ba \\ ca & 53 & 60 & 70 \\ b7 & d0 & e0 & e1 \\ 04 & 51 & e7 & 8c \end{bmatrix}.$$

Applying the row rotations gives:

$$\mathbf{State}_{SR} = \begin{bmatrix} 63 & 09 & cd & ba \\ 53 & 60 & 70 & ca \\ e0 & e1 & b7 & d0 \\ 8c & 04 & 51 & e7 \end{bmatrix}.$$

Serializing this State column by column produces:

$$\boxed{\texttt{6353e08c0960e104cd70b751bacad0e7}}$$

ShiftRows is fundamentally a fixed byte permutation. In hardware, it can normally be implemented by wiring rather than arithmetic logic. Students should avoid unnecessary sequential logic or multiplexing when a fixed interconnection is sufficient.

1.6 MixColumns and Finite-Field Arithmetic

The *MixColumns* transformation operates independently on each column of the State. For one column:

$$\begin{bmatrix} s'_0 \\ s'_1 \\ s'_2 \\ s'_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{bmatrix},$$

where all arithmetic is performed in $GF(2^8)$.

Addition in $GF(2^8)$ is bitwise XOR. Multiplication by 02_{16} can be implemented by the AES *xtime* operation:

$$\text{xtime}(x) = \begin{cases} (x \ll 1), & x_7 = 0, \\ (x \ll 1) \oplus 1B_{16}, & x_7 = 1, \end{cases}$$

where only the lower eight bits are retained. Multiplication by 03_{16} is:

$$03_{16} \cdot x = \text{xtime}(x) \oplus x.$$

For the running example, the ShiftRows output is:

$$\mathbf{State}_{SR} = \begin{bmatrix} 63 & 09 & cd & ba \\ 53 & 60 & 70 & ca \\ e0 & e1 & b7 & d0 \\ 8c & 04 & 51 & e7 \end{bmatrix}.$$

Consider the first column:

$$\begin{bmatrix} 63 \\ 53 \\ e0 \\ 8c \end{bmatrix}.$$

The first output byte is:

$$\begin{aligned} s'_0 &= (02 \cdot 63) \oplus (03 \cdot 53) \oplus e0 \oplus 8c \\ &= c6 \oplus f5 \oplus e0 \oplus 8c \\ &= 5f. \end{aligned}$$

Applying the same finite-field matrix multiplication to all four columns gives:

$$\mathbf{State}_{MC} = \begin{bmatrix} 5f & 57 & f7 & 1d \\ 72 & f5 & be & b9 \\ 64 & bc & 3b & f9 \\ 15 & 92 & 29 & 1a \end{bmatrix}.$$

Serializing the State column by column gives:

`5f72641557f5bc92f7be3b291db9f91a`

Consequently, MixColumns can be implemented using shifts, conditional XORs, and XOR networks rather than general-purpose integer multipliers. The selected organization should be considered when estimating critical path and hardware cost.

1.7 Round-Key XOR (AddRoundKey)

The AES standard names this transformation *AddRoundKey*. In hardware, the operation is a bit-wise XOR between the current State and the corresponding 128-bit round key. Therefore, it is written consistently with the pseudocode as:

$$\texttt{State} \leftarrow \texttt{State} \oplus K_r$$

or, in matrix form:

$$\mathbf{State}_{ARK} = \mathbf{State}_{MC} \oplus \mathbf{K}_r.$$

For Round 1 of the running example, the MixColumns output is:

$$\mathbf{State}_{MC} = \begin{bmatrix} 5f & 57 & f7 & 1d \\ 72 & f5 & be & b9 \\ 64 & bc & 3b & f9 \\ 15 & 92 & 29 & 1a \end{bmatrix}.$$

The first expanded round key

$$K_1 = \texttt{d6aa74fdd2af72fadaa678f1d6ab76fe}$$

is represented as:

$$\mathbf{K}_1 = \begin{bmatrix} d6 & d2 & da & d6 \\ aa & af & a6 & ab \\ 74 & 72 & 78 & 76 \\ fd & fa & f1 & fe \end{bmatrix}.$$

The round-key XOR therefore produces the Round-1 output State:

$$\mathbf{State}_{ARK} = \mathbf{State}_{MC} \oplus \mathbf{K}_1 = \begin{bmatrix} 89 & 85 & 2d & cb \\ d8 & 5a & 18 & 12 \\ 10 & ce & 43 & 8f \\ e8 & 68 & d8 & e4 \end{bmatrix}.$$

Serializing this State gives:

$$\boxed{\texttt{89d810e8855ace682d1843d8cb128fe4}}$$

which is the State at the end of Round 1.

In the final round, the same XOR operation uses K_{10} after SubBytes and ShiftRows. There is no MixColumns transformation in the final round.

This operation has small logic depth, but the architecture must ensure that the correct round key is available when the corresponding State reaches the XOR stage.

1.8 AES-128 Key Expansion

AES-128 uses the original 128-bit cipher key as K_0 and expands it into ten additional 128-bit round keys:

$$K_0, K_1, \ldots, K_{10}.$$

According to NIST FIPS 197, the AES-128 key schedule contains 44 words, each 32 bits wide:

$$w_0, w_1, \ldots, w_{43}.$$

The original key provides the first four words:

$$K_0 = [w_0\ w_1\ w_2\ w_3].$$

For AES-128, every following word is generated recursively as:

$$w_i = \begin{cases} w_{i-4} \oplus \mathrm{SubWord}(\mathrm{RotWord}(w_{i-1})) \oplus \mathrm{Rcon}[i/4], & i \bmod 4 = 0, \\ w_{i-4} \oplus w_{i-1}, & \text{otherwise.} \end{cases}$$

The two word transformations are:

$$\mathrm{RotWord}([a_0, a_1, a_2, a_3]) = [a_1, a_2, a_3, a_0]$$

and

$$\mathrm{SubWord}([a_0, a_1, a_2, a_3]) = [\mathrm{SBox}(a_0), \mathrm{SBox}(a_1), \mathrm{SBox}(a_2), \mathrm{SBox}(a_3)].$$

For AES-128, the round constants are:

$$\begin{aligned} \mathrm{Rcon}[1] &= \texttt{01000000}, & \mathrm{Rcon}[2] &= \texttt{02000000}, & \mathrm{Rcon}[3] &= \texttt{04000000}, \\ \mathrm{Rcon}[4] &= \texttt{08000000}, & \mathrm{Rcon}[5] &= \texttt{10000000}, & \mathrm{Rcon}[6] &= \texttt{20000000}, \\ \mathrm{Rcon}[7] &= \texttt{40000000}, & \mathrm{Rcon}[8] &= \texttt{80000000}, & \mathrm{Rcon}[9] &= \texttt{1b000000}, \\ \mathrm{Rcon}[10] &= \texttt{36000000}. \end{aligned}$$

Each round key is formed from four consecutive words:

$$\boxed{K_r = [w_{4r}\ w_{4r+1}\ w_{4r+2}\ w_{4r+3}]}, \qquad r = 0, \ldots, 10.$$

When represented as an AES round-key matrix, these four words become the four columns:

$$\mathbf{K}_r = \begin{bmatrix} w_{4r}[0] & w_{4r+1}[0] & w_{4r+2}[0] & w_{4r+3}[0] \\ w_{4r}[1] & w_{4r+1}[1] & w_{4r+2}[1] & w_{4r+3}[1] \\ w_{4r}[2] & w_{4r+1}[2] & w_{4r+2}[2] & w_{4r+3}[2] \\ w_{4r}[3] & w_{4r+1}[3] & w_{4r+2}[3] & w_{4r+3}[3] \end{bmatrix},$$

where $w_i[j]$ denotes byte j of word w_i from most significant to least significant byte.

For the running example:

$$K_0 = \texttt{000102030405060708090a0b0c0d0e0f},$$

so:

$$w_0 = \texttt{00010203}, \quad w_1 = \texttt{04050607},$$
$$w_2 = \texttt{08090a0b}, \quad w_3 = \texttt{0c0d0e0f}.$$

To generate the first word of K_1:

$$\begin{aligned} \text{RotWord}(w_3) = \text{RotWord}(\texttt{0c0d0e0f}) &= \texttt{0d0e0f0c}, \\ \text{SubWord}(\texttt{0d0e0f0c}) &= \texttt{d7ab76fe}, \\ \text{SubWord}(\text{RotWord}(w_3)) \oplus \text{Rcon}[1] &= \texttt{d6ab76fe}, \\ w_4 = w_0 \oplus \texttt{d6ab76fe} &= \texttt{d6aa74fd}. \end{aligned}$$

The remaining words of K_1 are then generated by the XOR chain:

$$w_5 = w_1 \oplus w_4 = \texttt{d2af72fa},$$
$$w_6 = w_2 \oplus w_5 = \texttt{daa678f1},$$
$$w_7 = w_3 \oplus w_6 = \texttt{d6ab76fe}.$$

Therefore:

$$K_1 = [w_4\ w_5\ w_6\ w_7] = \boxed{\texttt{d6aa74fdd2af72fadaa678f1d6ab76fe}},$$

which corresponds to:

$$\mathbf{K}_1 = \begin{bmatrix} \text{d6} & \text{d2} & \text{da} & \text{d6} \\ \text{aa} & \text{af} & \text{a6} & \text{ab} \\ \text{74} & \text{72} & \text{78} & \text{76} \\ \text{fd} & \text{fa} & \text{f1} & \text{fe} \end{bmatrix}.$$

The same recurrence is repeated to generate all subsequent round keys:

Round	**Round key**
0	000102030405060708090a0b0c0d0e0f
1	d6aa74fdd2af72fadaa678f1d6ab76fe
2	b692cf0b643dbdf1be9bc5006830b3fe
3	b6ff744ed2c2c9bf6c590cbf0469bf41
4	47f7f7bc95353e03f96c32bcfd058dfd
5	3caaa3e8a99f9deb50f3af57adf622aa
6	5e390f7df7a69296a7553dc10aa31f6b
7	14f9701ae35fe28c440adf4d4ea9c026
8	47438735a41c65b9e016baf4aebf7ad2
9	549932d1f08557681093ed9cbe2c974e
10	13111d7fe3944a17f307a78b4d2b30c5

Thus, the key-expansion logic must provide K_r at the same time as the corresponding State reaches the round-key XOR stage. The implementation may use one of the following approaches:

- **Precompute and store:** generate all round keys before data processing and store them
- **On-the-fly generation:** generate the next round key while encryption is in progress
- another architecture that is functionally equivalent and technically justified

The choice creates a trade-off among storage, combinational logic, latency, and control complexity. The key-expansion architecture must be included in the architecture exploration.

2 Requirements and Implementation Procedure

2.1 Requirements

The AES-128 accelerator is intended for a secure high-speed data link. The system requires a sustained encrypted payload throughput of at least 4.0 Gb/s. The AES core operates at a fixed target clock frequency of 200 MHz. Students must explore multiple microarchitectures and select the minimum-cost implementation that satisfies both the system throughput requirement and the implementation timing constraint.

The project focuses on AES-128 **encryption only**. AES decryption is not required.

The application-level throughput requirement is:

$$\boxed{TP_{required} \geq 4.0 \text{ Gb/s}}$$

The target implementation clock is:

$$f_{core} = 200 \text{ MHz}$$

which corresponds to:

$$\boxed{T_{clk,target} = 5 \text{ ns}}$$

Therefore, the synthesized implementation must achieve non-negative timing slack for a 5 ns clock period. Equivalently, the critical-path delay must satisfy:

$$\boxed{T_{critical} \leq 5 \text{ ns}}$$

All groups shall use the same ASIC implementation environment for synthesis and timing analysis. The project uses a 45-nm PDK and the standard-cell library condition shown in Table 1.

No.	Library Name	Temperature	Process Corner	VDD (V)	Threshold Type
1	`fast_vdd1v0_basicCells_hvt.lib`	0 °C	Fast (FF)	1.1	HVT (High-V_t)

Table 1: Standard-cell library condition for synthesis and timing analysis

All reported area, critical-path delay, maximum clock frequency, and timing results must be obtained using this implementation condition unless otherwise specified by the instructor.

Since each encrypted block contains 128 bits, the required block rate is:

$$R_{block,required} = \frac{4.0 \times 10^9}{128} = 31.25 \text{ Mblocks/s}$$

Therefore, at 200 MHz:

$$II_{block,target} \leq \frac{200}{31.25} = 6.4$$

Because the initiation interval is an integer number of cycles, the implemented architecture must satisfy:

$$\boxed{II_{block} \leq 6}$$

The primary optimization objective is:

$$\boxed{\text{minimize standard-cell area subject to timing and throughput constraints}}$$

Students are not given a fixed number of S-boxes, round units, pipeline stages, or key-storage elements. These are architecture decisions that must be explored and justified.

2.2 Top-Level Interface

The required top-level ports are described in Table 2.

Name	Port Type	Width	Description
i_clk	Input	1 bit	Core clock.
i_rst_n	Input	1 bit	Active-low reset.
i_key_valid	Input	1 bit	Indicates that a new AES-128 key is presented.
o_key_ready	Output	1 bit	Indicates that the design can accept a new key.
i_key	Input	128 bits	AES-128 cipher key.
i_data_valid	Input	1 bit	Indicates that a plaintext block is valid.
o_data_ready	Output	1 bit	Indicates that a plaintext block can be accepted.
i_plaintext	Input	128 bits	Plaintext block.
o_cipher_valid	Output	1 bit	Indicates that the ciphertext output is valid.
i_cipher_ready	Input	1 bit	Indicates that the receiver can accept the ciphertext.
o_ciphertext	Output	128 bits	Ciphertext block.

Table 2: Required AES-128 top-level input and output ports

A key is accepted only when:

$$i_key_valid \wedge o_key_ready = 1.$$

A plaintext block is accepted only when:

$$i_data_valid \wedge o_data_ready = 1.$$

A ciphertext block is transferred only when:

$$o_cipher_valid \wedge i_cipher_ready = 1.$$

The design must not lose or duplicate data when the output is stalled by $i_cipher_ready = 0$.

2.3 AES-128 Hardware Architecture Exploration

AES-128 provides several natural hardware architectures.

2.3.1 Iterative Architecture

An iterative architecture reuses one round datapath over multiple cycles.

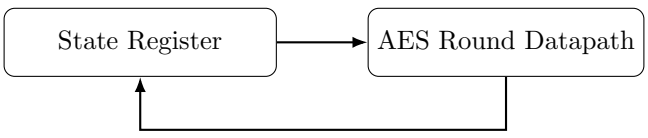


Figure 1: Conceptual iterative AES architecture

The main advantage is lower area. The main disadvantage is a larger block initiation interval because a hardware round is reused.

2.3.2 Unrolled Architecture

Multiple AES rounds can be instantiated spatially:

$$Round_1 \rightarrow Round_2 \rightarrow \cdots \rightarrow Round_{10}.$$

Unrolling reduces temporal reuse of the round hardware and can improve throughput, but increases area and may create a long combinational path.

2.3.3 Pipelined Architecture

Pipeline registers can be inserted between AES rounds, or within a round if technically justified. A fully unrolled and properly pipelined implementation may accept a new 128-bit block every cycle:

$$II_{block} = 1.$$

However, this architecture has a substantially larger register and logic cost. Therefore, **the architecture with the highest throughput is not automatically the best architecture**. The selected architecture must satisfy the project constraints with the lowest justified hardware cost.

2.4 Throughput and Initiation Interval

If a 128-bit block can be accepted every II_{block} core clock cycles, the sustained encryption throughput is:

$$TP_{AES} = \frac{128\, f_{core}}{II_{block}} \quad [\text{bit/s}].$$

For example, at $f_{core} = 200$ MHz and $II_{block} = 10$:

$$TP_{AES} = \frac{128 \times 200 \times 10^6}{10} = 2.56 \text{ Gb/s}.$$

This project requires students to use throughput, timing, and hardware-cost analysis to select an AES-128 microarchitecture rather than implementing a predefined architecture.

2.5 Implementation Procedure

2.5.1 Step 1 – Estimate Candidate Architectures

Before writing the final RTL, each group must investigate at least three candidate AES architectures. The purpose of this step is to estimate the resource requirement and expected performance of each candidate before committing to a detailed RTL implementation.

The candidates must include different degrees of spatial and/or temporal resource sharing. Possible candidates include:

- iterative round architecture
- partially unrolled architecture
- fully unrolled architecture
- pipelined variants of the above architectures

For each candidate, estimate at least:

- number of AES round datapaths or equivalent round resources

- number of physical S-box units
- number of physical MixColumns units
- key-expansion organization
- number and location of major pipeline stages
- block latency in cycles
- block initiation interval II_{block}
- expected throughput at 200 MHz
- expected critical-path trend
- expected hardware-area trend

The estimation shall be based on the selected resource organization, operation scheduling, and the known AES-128 computation flow. Exact cell area and timing are not required at this stage.

A comparison table similar to Table 3 shall be included in the report.

Candidate	Round Units	S-box Units	MixColumns Units	Key Expansion	Pipeline Stages	Latency	II_{block}	TP	Timing/Area Trend
A									
B									
C									

Table 3: Pre-synthesis architecture estimation and comparison

2.5.2 Step 2 – Select the Architecture and Resource Budget

Using the estimates from Step 1, each group shall first eliminate candidates that cannot reasonably satisfy the project constraints:

$$II_{block} \leq 6$$

and:

$$T_{critical} \leq 5 \text{ ns}.$$

Among the estimated-feasible candidates, the group shall select the architecture with the lowest justified hardware cost. The selected architecture shall define a concrete resource budget before final datapath design begins, including:

- number of round datapaths
- number of S-box units
- number of MixColumns units
- key-expansion organization
- state and round-key storage organization
- major pipeline boundaries

The architecture selected at this stage is a design hypothesis based on pre-synthesis estimation. It must later be validated by RTL synthesis and timing analysis. If the measured implementation does not satisfy the constraints, the group must revise the architecture and repeat the design flow.

2.5.3 Step 3 – Design the Datapath and Control Unit

After the resource budget has been selected, the group shall design the detailed datapath and control unit for that architecture. The design shall not assume unlimited functional resources. All AES operations must be mapped onto the physical resources selected in Step 2.

The datapath and controller shall implement the same encryption sequence defined in Section 1.3:

- initial round-key XOR: `State` $\leftarrow$ `Plaintext` $\oplus K_0$
- main loop for $r = 1, \ldots, 9$: SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ round-key XOR with K_r
- final round: SubBytes $\rightarrow$ ShiftRows $\rightarrow$ round-key XOR with K_{10}, with no MixColumns

The report shall show how the selected physical resources are used over time. If a resource is shared, the schedule and control required for that sharing must be explicitly documented.

For SubBytes, if fewer than 16 physical S-box units are selected for a 128-bit State, the design must show how the 16 byte substitutions are scheduled across the available S-box units.

For MixColumns, the design must show how the four State columns are mapped onto the selected number of physical MixColumns units. Generic integer multipliers shall not be used to implement multiplication by 02_{16} and 03_{16}. The required $GF(2^8)$ arithmetic shall use XOR and the AES *xtime* operation or another equivalent optimized structure.

ShiftRows should be implemented as a byte permutation unless another implementation is justified.

The control unit shall generate all control required by the selected datapath, including where applicable:

- round index
- operation sequencing
- resource-select signals
- State-register write enables
- round-key selection
- pipeline valid propagation
- input-ready and output-valid behavior

Students shall document the datapath hierarchy and the control-unit behavior from the top module down to the major RTL blocks.

2.5.4 Design the Key Expansion

The design shall support a programmable 128-bit AES key and shall generate or provide the round keys K_0 through K_{10} according to the AES-128 key expansion described in Section 1.8.

Students may:

- precompute and store all 11 round keys
- generate round keys on-the-fly
- use another equivalent organization

For every encrypted block, the architecture must guarantee that K_0 is available for the initial round-key XOR, K_r is available for main round $r = 1, \ldots, 9$, and K_{10} is available for the final round-key XOR.

The selected key-expansion architecture must be analyzed in terms of:

- storage requirement
- key-setup latency
- interaction with the encryption pipeline
- hardware cost
- timing

A newly accepted key shall not corrupt a block that is currently being encrypted. The protocol used for key updates must be documented.

2.5.5 Pipelining and Control

Students must determine whether pipelining is required to satisfy the target clock period.

If pipeline registers are inserted, the report must identify:

- the location of each pipeline boundary
- the corresponding block latency
- the resulting block initiation interval
- the control and valid-bit propagation through the pipeline

The controller must preserve the association between each data block and the correct round key.

2.5.6 Streaming Interface and Backpressure

The plaintext and ciphertext interfaces use ready-valid flow control.

The design must support arbitrary pauses on the output interface. When $i_cipher_ready = 0$, the design shall preserve all unconsumed output data.

If the selected architecture cannot immediately stop all internal pipeline stages, students must design sufficient buffering or an equivalent flow-control mechanism to prevent data loss.

The sustained throughput reported for the architecture shall be measured under continuous input traffic with no output stalls.

2.5.7 Simulation and Basic Functional Verification

Milestone 1 evaluates digital design. Therefore, a basic functional testbench is required, but advanced verification methodology is reserved for the verification milestone.

Students must:

- test the design using official or otherwise traceable AES-128 known-answer test vectors
- test multiple plaintext blocks using the same key
- test at least two different keys
- test output backpressure
- confirm correct behavior after reset
- confirm that no block is lost, duplicated, or reordered

At least one standard AES-128 test vector used in the report shall identify the source of the vector.

All simulation programs shall be written in SystemVerilog and executed using the course simulation tool.

2.5.8 Synthesis and Timing Analysis

Students shall synthesize the final RTL using the 45-nm PDK and the standard-cell library condition specified in Table 1.

The synthesis report shall include:

- total cell area or equivalent implementation cost
- critical path delay
- achieved maximum clock frequency
- register count
- combinational cell count where available
- timing violations, if any

The final implementation must demonstrate non-negative timing slack at the specified 200 MHz target clock and sustained AES throughput of at least:

$$\boxed{TP_{AES} \geq 4.0 \text{ Gb/s}}$$

The final report shall compare the pre-synthesis estimates with the measured implementation results for the selected architecture, as illustrated in Table 4.

Metric	Estimated	Measured	Requirement
Cell area			Minimize
Critical-path delay			≤ 5 ns
Maximum clock frequency			≥ 200 MHz
Block latency			–
II_{block}			≤ 6
AES throughput			≥ 4.0 Gb/s

Table 4: Estimated and measured results for the selected architecture

If the selected architecture fails either the timing constraint or the system throughput requirement, the group must revise the resource allocation, datapath, pipelining, or control strategy and repeat synthesis.

2.5.9 Optional FPGA Demonstration

If FPGA hardware is available, groups may demonstrate the AES-128 accelerator on the supported course FPGA platform. The FPGA demonstration is optional unless otherwise announced by the instructor.

An FPGA implementation does not replace ASIC synthesis and timing analysis.

2.5.10 Reports and Design Defense

All groups must submit a report and conduct a design defense with the instructor.

The report must clearly present:

- AES-128 functional decomposition following the initial XOR, nine-round main loop, and final round
- pre-synthesis architecture and resource estimates
- architecture-selection rationale and selected resource budget
- datapath and control unit derived from the selected resource budget
- S-box organization
- MixColumns implementation
- key-expansion architecture
- pipeline structure
- ready-valid behavior and output buffering
- RTL hierarchy
- synthesis and timing results
- measured latency, initiation interval, and throughput
- comparison between predicted and measured design metrics

For each major RTL block, provide:

- a block diagram
- input and output ports
- signal widths
- signal descriptions
- the purpose of the block

During the design defense, each student must be able to explain the architecture and the RTL implemented by the group.

3 Grading Criteria

3.1 Design Learning Outcomes Assessed

This milestone primarily evaluates the digital-design learning outcomes shown in Table 5.

Learning Outcome	Evidence in the Project
L.O.1.1	Architecture exploration, pipelining, parallelism, resource sharing, initiation interval, throughput, and timing analysis.
L.O.1.2	AES finite-field arithmetic datapath, MixColumns optimization, S-box organization, and critical-path optimization.
L.O.2.1	Key storage, pipeline storage, buffering, ready-valid flow control, and streaming bandwidth analysis.

Table 5: Learning-outcome mapping for Milestone 1

The primary expected design flow is:

Specification → Architecture Exploration → RTL Design → Synthesis → Constraint Verification

The project is evaluated primarily on architecture design and implementation rather than only functional correctness.

Criteria	Grade
Architecture exploration and justified selection under throughput, timing, and hardware-cost constraints	2.5 pts
AES datapath design: SubBytes, ShiftRows, MixColumns, round-key XOR, and resource organization	2.0 pts
Key expansion, pipelining, controller, and ready-valid data flow	1.5 pts
Functionally correct synthesizable RTL and basic simulation	1.5 pts
Synthesis, timing closure, and demonstrated throughput target	1.5 pts
Technical report and individual design defense	1.0 pt
Total	**10 pts**

Table 6: Grading Criteria